Food Store en la feria: la base de la tablet y el cierre de la capa de datos

TP N.º 8 · Unidad 8 · Bases de Datos II · Tecnicatura Universitaria en Programación · UTN — **Ignacio Martín Rodríguez** — 02/11/2026 · PostgreSQL 16.15 (base de pruebas **copia_trabajo**) · SQLite 3.53 · repositorio **github.com/Nacho30/UTN-2026-BASE-DE-DATOS**, rama **tp-unidad-8**, etiqueta **tp8**, carpeta **food-store-tp-u8/**

Regla de oro: el móvil propone, el servidor dispone. La tablet sólo consulta y arma borradores; precio, stock y ventas los decide PostgreSQL. **SIMULADO_APROBADO no es una venta:** ningún paso de la tablet escribe en el servidor.

1.a Qué datos viajan a la tablet

Dato en PostgreSQL	¿Viaja?	Cómo llega a SQLite / por qué no
producto.id_producto	Sí	**id** de **producto_cache**; lo usan las FK de los ítems y el semáforo
producto.nombre	Sí	**nombre**, para mostrar y para el motivo del semáforo
producto.precio NUMERIC(10,2)	Sí	**precio_centavos** entero: precio × 100, exacto
producto.stock	Sí	**stock_visto**: sólo informativo, envejece en cuanto se copia
producto.activo, categoria.activo	Sí, derivados	**disponible** (activo, categoría activa y stock > 0) y **eliminado** (= NOT activo, lápida)
fecha de la copia	Sí	**copiado_en** = **fecha_snapshot** de la vista
producto.descripcion, id_categoria, created_at	No	La venta en la feria no los usa
cliente (nombre, apellido, email, teléfono)	No	Datos personales: la tablet registra sólo un alias de vendedor
pedido, pedido_producto, usuarios y credenciales (U5)	No	Historial y seguridad viven sólo en el servidor

Viaja lo mínimo para vender sin señal: identificar el producto, mostrar su nombre y su precio y saber si se puede ofrecer. **La fuente de verdad es PostgreSQL**: la tablet tiene una copia fechada que puede estar vieja, por eso el precio y el stock que ve son sólo una referencia y el semáforo vuelve a compararlos con una instantánea nueva.

1.b, 1.c y 2.a En dos líneas

schema_sqlite.sql empieza con **PRAGMA foreign_keys = ON;** y crea las cuatro tablas pedidas más **snapshot_servidor**, todas **STRICT**, con centavos enteros, **CHECK** de dominios y cantidades, FK y un trigger que impide cambiar una solicitud ya evaluada. **pruebas_sqlite.sql** guarda un borrador completo con **COMMIT**, deshace otro con **ROLLBACK**, lee el catálogo sólo desde **producto_cache** (C1; repetido con PostgreSQL detenido en **14_c1_sin_servidor.txt**) y muestra que cantidad 0 y producto 999 fallan sin dejar filas (C7). La vista **v_snapshot_movil** incluye los productos dados de baja (lápidas); la instantánea inicial es del **2026-11-02T13:50:18Z** (UTC).

2.c Los casos

Cada caso: borrador en la tablet → cambio en **copia_trabajo** → **exportar_snapshot.sql** → **cargar_snapshot.sql** → **semaforo.sql**. Todo lo corre **correr_todo.sh** desde cero; las salidas están en **docs/evidencia/03** a **08**. Las tablas de abajo resumen esas salidas.

Caso	Comandos	Resultado	Explicación
C2	c2_borrador.sql (2 × Hamburguesa clásica a 8.500,00) · servidor/casos/c2_subir_precio.sql (precio 9.200,00) · exportar, cargar, semaforo.sql · c2_acepta.sql · semaforo.sql	CONFLICTO: «precio visto 8500.00, precio actual 9200.00». La solicitud nueva: SIMULADO_APROBADO	No hay causa roja, pero el precio cambió. La solicitud en CONFLICTO queda como historial (estado final); el cliente acepta, el borrador toma el precio del catálogo nuevo y se encola otro UUID.
C3	c3_borrador.sql (5 × Hamburguesa doble; la tablet ve 9) · c3_ajustar_stock.sql (quedan 3) · exportar, cargar, semaforo.sql	RECHAZADO: «stock insuficiente, pide 5 y quedan 3»	La tablet no valida stock: el que ve es viejo. El rojo lo da la instantánea nueva. El stock bajó por un ajuste preparado en el servidor, no por la tablet.
C4	c4_borrador.sql (2 × Agua mineral, 1 × Gaseosa) · c4_baja_producto.sql (activo = false) · exportar, cargar, semaforo.sql · c4_catalogo.sql	RECHAZADO: «Agua mineral 500 ml: dado de baja». El catálogo ya no la muestra; la fila queda con eliminado = 1	La baja viaja como lápida: si se borrara la fila, la FK del ítem del borrador quedaría huérfana. Un borrador nuevo ya no puede incluirla (0 ítems insertados).
C5	c5_borrador.sql (Pizza muzzarella y Flan) · exportar y cargar sin cambios · semaforo.sql	SIMULADO_APROBADO: «todo coincide con la instantánea»	Verde = coincide con la instantánea. No crea un pedido ni descuenta stock (ver verificación final).

Caso	Comandos	Resultado	Explicación
C6	c6_reenvio.sql (mismo UUID de C5) · semaforo.sql otra vez	«UNIQUE constraint failed: cola_sync.solicitud_uuid»; el UPDATE para forzar PENDIENTE lo frena el trigger; 0 solicitudes evaluadas	El UUID es la clave de idempotencia: no se duplica y repetir la simulación no reevalúa porque sólo toma PENDIENTE.

C6 en un sistema de verdad (parada 8): la protección no puede depender de la tablet. El servidor guardaría cada **solicitud_uuid** en una tabla con **UNIQUE** y lo insertaría en la **misma transacción** que crea el pedido (**INSERT ... ON CONFLICT DO NOTHING**); si el UUID ya estaba, devuelve el resultado anterior sin volver a vender.

Semáforo al final de todos los casos (**07_caso_c6.txt**):

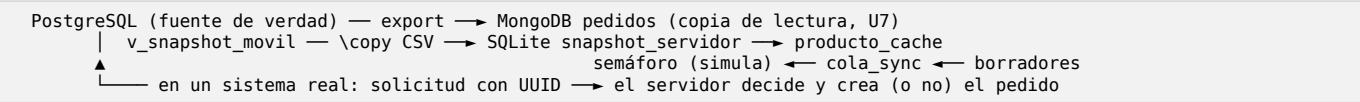
```
solicitud  borrador  estado             motivo
---------  --------  -----------------  ------------------------------------------------------------------------------
51…0001    b1…0001   SIMULADO_APROBADO  todo coincide con la instantánea
52…0001    b2…0002   CONFLICTO          producto 1 (Hamburguesa clásica): precio visto 8500.00, precio actual 9200.00
52…0002    b2…0002   SIMULADO_APROBADO  todo coincide con la instantánea
53…0001    b3…0003   RECHAZADO          producto 2 (Hamburguesa doble): stock insuficiente, pide 5 y quedan 3
54…0001    b4…0004   RECHAZADO          producto 5 (Agua mineral 500 ml): dado de baja
55…0001    b5…0005   SIMULADO_APROBADO  todo coincide con la instantánea
```

PostgreSQL no cambió por la simulación (**servidor/verificacion_final.sql**, **08_verificacion_final.txt**): siguen los 4 pedidos y 6 líneas, y el servidor es idéntico a la última instantánea, exportada antes de C5 y C6: los verdes no descontaron nada. Los únicos cambios son los tres preparados a mano (precio del 1, stock del 2, baja del 5).

```
 pedidos | lineas | ultimo_id_pedido
---------+--------+------------------
       4 |      6 |                4
(1 row)
 diferencia | id | nombre | precio_centavos | stock | disponible | eliminado
------------+----+--------+-----------------+-------+------------+-----------
(0 rows)
 id_producto |       nombre        |  precio  | stock | activo
-------------+---------------------+----------+-------+--------
           1 | Hamburguesa clásica |  9200.00 |    18 | t
           2 | Hamburguesa doble   | 11000.00 |     3 | t
           3 | Pizza muzzarella    | 12000.00 |     7 | t
           4 | Gaseosa 500 ml      |  2500.00 |    48 | t
           5 | Agua mineral 500 ml |  1800.00 |    39 | f
           6 | Flan casero         |  3500.00 |     3 | t
           7 | Pizza napolitana    | 13000.00 |     0 | f
(7 rows)
```

3.a Mapa de la capa de datos

Motor	Qué guarda	Quién manda	Cómo se actualiza
PostgreSQL (U1–U6, U8)	Catálogo, precios, stock, clientes, pedidos y líneas, usuarios y auditoría	**Manda sobre todo.** Único lugar donde se crea una venta y se descuenta stock (trigger RN1/RN2 con FOR UPDATE)	Transacciones de la app; respaldo diario (U6)
MongoDB (U7)	Colección **pedidos**: un documento por pedido con cliente y líneas embebidas	No manda: copia de lectura derivada de PostgreSQL	Se regenera desde PostgreSQL (export + insertMany)
SQLite (U8, tablet)	**producto_cache** (copia del catálogo), borradores, ítems y **cola_sync**	Catálogo: manda PostgreSQL. Borradores: la tablet, hasta que el servidor los evalúa. Estados de la cola: el semáforo, sobre la instantánea	Instantánea CSV de **v_snapshot_movil**; los borradores, el vendedor



3.b Plan de mantenimiento (agregado al plan de la Unidad 6)

Pregunta	Respuesta
¿Qué se respalda y cómo se prueba la restauración?	**PostgreSQL**: igual que en U6, **pg_dump -Fc** diario con verificación y simulacro de restauración en una base aparte (**restore_test.sh**), ahora con **v_snapshot_movil** y la migración 002. **Tablet**: el archivo **foodstore_movil.db** se copia con **VACUUM INTO** al cerrar cada jornada, porque los borradores PENDIENTE sólo existen ahí; se prueba abriendo la copia y corriendo **PRAGMA integrity_check** y **PRAGMA foreign_key_check**. **MongoDB** no se respalda: se regenera desde PostgreSQL.

Pregunta	Respuesta
Si se restaura un respaldo viejo de PostgreSQL, ¿qué hacen las tablets?	Recargan el catálogo completo con una instantánea nueva antes de vender. **cargar_snapshot.sql** ya lo contempla: un producto que el servidor dejó de informar no se borra (lo usan borradores), queda **disponible = 0**, y una solicitud con ese producto sale RECHAZADA (**11_respaldo_viejo.txt**). Las solicitudes ya evaluadas no se reenvían solas: las ventas reales perdidas (RPO) se reconstruyen desde la cola, y el UUID evita duplicarlas si el servidor sí las tenía.
¿Cada cuánto y quién revisa CONFLICTO y RECHAZADO?	**CONFLICTO**: el vendedor, en el momento, con el cliente (acepta el precio nuevo → solicitud nueva). **RECHAZADO**: el encargado del food truck, al cierre de cada jornada de feria, con la consulta de **cola_sync** por estado; decide si se ofrece un reemplazo o se anula el borrador. Lo PENDIENTE de más de un día se revisa antes de la jornada siguiente.

3.c Code review con IA

Revisión de los scripts de la cursada hecha con Claude en el rol del agente Plan de OpenCode (ver DUIA). Cada hallazgo se comprobó en una base descartable, antes y después de corregir (**revision/verificar_hallazgos.sh → 09_code_review.txt**; tablet → **10**, **11** y **13**). Las pruebas de 001 y de los programables dan la misma salida antes y después de las correcciones.

#	Unidad · script	Hallazgo	Decisión	Motivo y comprobación
H1	TPI · programables	**sp_registrar_pedido** saltea en silencio un id_producto inexistente: el INSERT ... SELECT sin filas no es error	**Corregido**	Carrito de 2 productos (uno es 999) → pedido con 1 línea. Ahora **IF NOT FOUND THEN RAISE** y no se crea el pedido.
H2	U2 · migraciones/001	El trigger RN1/RN2 sólo corre en INSERT: un UPDATE de la línea cambia cantidad sin tocar stock o la pasa a un producto de baja	**Corregido**	UPDATE cantidad 2 → 40: stock del flan sigue en 3. Migración **002** bloquea el UPDATE; la cancelación con CASCADE sigue andando (**12_pruebas_002.txt**).
H3	U1 · schema.sql	Falta un índice sobre **pedido_producto(id_pedido)** para los joins y el CASCADE	**Descartado**	La PK **(id_pedido, id_producto)** lo tiene como columna líder: EXPLAIN usa **pedido_producto_pkey**. Otro índice sólo encarecería las escrituras.
H4	U2 · lab/reset.sql	Dice «solo en copia_trabajo» pero corre en cualquier base	**Corregido**	Corrido sobre otra base: stock total 124 → 133. Guarda con **DO** + **ON_ERROR_STOP** (sin esto último psql seguía tras el error).
H5	U2 · lab/reset.sql	UPDATE de producto sin WHERE, contra la regla de AGENTS.md	**Descartado**	Es el reset deliberado de los 7 productos del laboratorio; la regla apunta a UPDATE accidentales. Con H4 sólo puede correr en copia_trabajo.
L1	U8 · semaforo.sql	Evalúa cada solicitud por separado: dos que juntas superan el stock salen verdes	**Aceptado**	Limitación documentada. 2 + 2 flanes con stock 3 → dos verdes; en el servidor el segundo **sp_registrar_pedido** falla por RN2. Reservar stock en la tablet sería decidir por el servidor.
P1	U8 · scripts de movil/	**PRAGMA foreign_keys** es por conexión: un script que no lo activa inserta ítems de productos inexistentes	**Aceptado**	Sin el PRAGMA, el ítem con producto 999 se inserta; **foreign_key_check** lo marca. Todos los scripts empiezan con el PRAGMA (**13_pragma_por_conexion.txt**).

Desarrollo de tres hallazgos

H1 (TPI, corregido). En **revision_antes**, dentro de **BEGIN ... ROLLBACK**, **CALL sp_registrar_pedido(1, 'EFECTIVO', '[{"id_producto": 4, ...}, {"id_producto": 999, ...}]')** devolvió el pedido 5 con **1 línea registrada de 2 pedidas**: el cliente pagaría por algo que nunca se registró. Es el procedimiento que recibiría las solicitudes de la tablet, así que el error también afectaba a U8. Con la corrección, el mismo CALL falla con «El producto 999 no existe» y el pedido no se crea; un carrito válido sigue funcionando.

H2 (U2, corregido). Con **UPDATE pedido_producto SET cantidad = 40** sobre la línea del flan, la cantidad pasó a 40 y el stock siguió en 3: RN2 no se cumple. También se pudo pasar una línea al producto 7, dado de baja (RN1). La migración **002** agrega un trigger **BEFORE UPDATE** que rechaza el cambio: una línea se corrige cancelando el pedido (**sp_cancelar_pedido** repone stock) y registrándolo de nuevo. No se bloqueó DELETE porque el CASCADE de **pedido** lo necesita; la prueba 002 muestra que cancelar sigue funcionando y repone el stock (3 → 5).

H3 (U1, descartado con fundamento). La sugerencia de indexar **pedido_producto(id_pedido)** suena razonable porque es la FK que se recorre en cada join y en el CASCADE. Pero un B-tree sobre **(id_pedido, id_producto)** sirve para buscar por su primera columna: con **enable_seqscan = off**, el EXPLAIN de **WHERE id_pedido = 3** usa **Bitmap Index Scan on pedido_producto_pkey**. Un índice extra duplicaría la PK y sumaría costo a cada INSERT, como se midió en U3 con el índice descartado sobre **forma_pago**.